Deep Neural Networks

Unit 2 — Architecture, Activation Functions & Hyperparameter Tuning

1 Deep Learning and Its Architectures

1.1 What Deep Learning Is and Why Depth Matters

Deep Learning is a subfield of machine learning that uses artificial neural networks with *multiple hidden layers* to learn hierarchical representations of data directly from raw inputs, without requiring hand-engineered features.

A traditional machine learning pipeline requires a human expert to decide which features matter — edge histograms for images, n-grams for text, and so on. Deep learning replaces this manual step with a stack of learned transformations. Each layer learns to transform its input into a slightly more abstract representation, and successive layers compose those abstractions.

1.1.1 Representation Learning and Feature Hierarchy

The core idea is that raw data has latent structure at multiple levels of abstraction. In image recognition:

- **Layer 1** detects local patterns: edges, blobs, colour transitions.
- **Layer 2** combines edges into textures and corners.
- **Layer 3** combines textures into object parts (eyes, wheels).
- **Layer 4+** combines parts into full objects (faces, cars).

This hierarchy is *learned from data*, not programmed by hand. The same principle holds for audio (samples → phonemes → words → meaning) and text (characters → words → phrases → semantics).

1.1.2 Why Deep Beats Wide

A theoretically infinite-width single hidden layer can approximate any continuous function (Universal Approximation Theorem). So why add depth?

- **Exponential representational efficiency.** Functions that require exponentially many neurons to represent with one layer can be represented with polynomially many neurons across multiple layers. Depth allows *reuse* of sub-computations.
- **Better generalisation.** Hierarchical features generalise across tasks. Features learned for dog recognition partially transfer to cat recognition.
- **Practical learnability.** Gradient descent finds good solutions more reliably in deep networks because the loss landscape, while complex, has been empirically shown to have fewer poor local minima than theory predicts.

Intuition — why depth helps: Think of recognising a face. A wide-but-shallow network has to memorise every face it has seen as a pattern. A deep network instead learns to first detect eyes, then that two eyes above a nose makes a face — a rule that generalises to millions of unseen faces. Depth allows the network to build concepts from simpler sub-concepts, just as humans do.

Common misconception — deeper is always better. Adding layers beyond what the problem requires causes overfitting, slower training, and vanishing gradients. Depth is a design choice, not a default setting. More data, proper regularisation, and the right architecture matter more than raw depth.

1.2 DNN vs. Shallow Networks

Dimension	Shallow Network (1–2 hidden layers)	Deep Network (3+ hidden layers)
Feature engineering	Requires manual features	Learns features from raw data
Parameter efficiency	Needs exponentially more neurons for same expressiveness	Shares sub-computations across layers
Training difficulty	Simpler, fewer gradient issues	Vanishing/exploding gradients are real risks
Data requirements	Can work on small datasets	Typically needs large datasets
Interpretability	Relatively easier to inspect	Much harder to interpret (black-box concern)
Task suitability	Tabular data, small-scale tasks	Images, audio, text, sequences
Convergence	Faster to converge	Slower; needs careful initialisation & optimisers

1.3 Feedforward Networks / MLPs — The Base Architecture

A **Feedforward Neural Network** (also called a Multilayer Perceptron, MLP) is the foundational deep learning architecture. Information flows strictly forward: from input layer → hidden layers → output layer, with no cycles or feedback connections.

Structure:

- **Input layer:** receives the raw feature vector $\mathbf{x} \in \mathbb{R}^n$.
- **Hidden layers:** each neuron computes $z = \mathbf{w}^\top \mathbf{x} + b$, then applies a non-linear activation $a = \sigma(z)$.
- **Output layer:** produces predictions; activation depends on task (sigmoid for binary, softmax for multi-class, linear for regression).

Forward pass (single layer):

$$\mathbf{z}^{(l)} = \mathbf{W}^{(l)}\,\mathbf{a}^{(l-1)} + \mathbf{b}^{(l)}, \qquad \mathbf{a}^{(l)} = \sigma\big(\mathbf{z}^{(l)}\big)$$

Training: parameters $\mathbf{W}$ and $\mathbf{b}$ are learned by minimising a loss function via backpropagation and gradient descent.

Advantages: Universal approximator in theory; well-understood; forms the backbone of all other architectures.

Limitations: Does not exploit spatial structure (unlike CNN) or sequential structure (unlike RNN); scales poorly with raw image/text inputs; prone to overfitting on small data.

1.4 Autoencoders

An **Autoencoder** is a neural network trained to *reconstruct its own input*. It consists of an **encoder** that compresses the input into a compact latent representation, and a **decoder** that reconstructs the original input from that representation.

1.4.1 Architecture

$$\underbrace{\mathbf{x}}_{\text{input}} \xrightarrow{\text{Encoder } f_\theta} \underbrace{\mathbf{z}}_{\text{latent code}} \xrightarrow{\text{Decoder } g_\phi} \underbrace{\hat{\mathbf{x}}}_{\text{reconstruction}}$$

- **Encoder** f_θ: a stack of layers that progressively reduces dimensionality. Maps $\mathbf{x} \in \mathbb{R}^n$ to $\mathbf{z} \in \mathbb{R}^k$ where $k \ll n$.
- **Latent space** $\mathbf{z}$: the bottleneck. The network is forced to learn only the most informative features to survive this compression.
- **Decoder** g_ϕ: mirrors the encoder, expanding $\mathbf{z}$ back to input dimensionality.

1.4.2 Loss Function

For continuous inputs (e.g., images), the standard loss is **Mean Squared Error** (reconstruction loss):

$$\mathcal{L} = \frac{1}{n}\sum_{i=1}^{n}(x_i - \hat{x}_i)^2$$

For binary inputs, Binary Cross-Entropy is used instead.

1.4.3 Why It Works

The bottleneck constraint forces the encoder to discard noise and redundancy, retaining only the signal needed to reconstruct the input. The learned latent space captures the *underlying structure* of the data, not its raw pixel/token values.

Intuition: Imagine compressing a high-resolution photo to a 100-word description and then asking someone to redraw it from just those words. To keep the reconstruction accurate, your 100 words must capture what actually matters — shapes, colours, composition — not random noise. The autoencoder's latent vector is that 100-word description, and the network must figure out the best way to describe the data.

1.4.4 Variants

- **Denoising Autoencoder:** Input is corrupted with noise; the model is trained to reconstruct the *clean* original. Forces the latent space to be more robust and meaningful.
- **Sparse Autoencoder:** A regularisation penalty forces most latent activations to be near zero, encouraging the network to learn disentangled features.
- **Variational Autoencoder (VAE):** Encoder outputs a probability distribution $\mathcal{N}(\mu, \sigma^2)$ over the latent space rather than a point. Enables *generation* of new samples by sampling from the distribution.

1.4.5 Applications

- **Dimensionality reduction:** Non-linear alternative to PCA. Useful for visualisation and feature extraction.
- **Anomaly detection:** Train on normal data. Anomalous inputs reconstruct poorly (high loss), making reconstruction error a reliable anomaly score.
- **Denoising:** Use the denoising variant to remove noise from images or signals.
- **Generative modelling:** VAEs generate new, plausible samples from the learned data distribution.
- **Data compression:** Store only the latent code $\mathbf{z}$; decode on demand.

1.4.6 Failure Modes and Limitations

- If the latent space is too large, the network may simply memorise inputs without learning meaningful structure.
- If it is too small, reconstruction quality degrades badly.
- Autoencoders are not generative by default — a standard autoencoder's latent space has no principled structure for sampling new data points (unlike VAEs).
- Sensitive to the choice of architecture, bottleneck size, and regularisation.

1.5 Generative Adversarial Networks (GANs)

A **GAN** consists of two networks trained in opposition: a **Generator** G that produces synthetic data, and a **Discriminator** D that tries to distinguish real data from generated data. The generator improves by fooling the discriminator; the discriminator improves by not being fooled.

1.5.1 Architecture and Training Loop

$$\mathbf{z} \sim p(\mathbf{z}) \xrightarrow{G} \hat{\mathbf{x}} = G(\mathbf{z})$$

$$D(\mathbf{x}) \to [0, 1] \quad \text{(probability that } \mathbf{x} \text{ is real)}$$

Training alternates between two steps:

1. **Discriminator step:** Fix G. Train D to output $D(\mathbf{x}) \approx 1$ for real data and $D(G(\mathbf{z})) \approx 0$ for generated data.

2. **Generator step:** Fix D. Train G to make $D(G(\mathbf{z})) \approx 1$ — i.e., to produce outputs that fool D.

1.5.2 GAN Loss

The original GAN minimax objective:

$$\min_G \max_D \; \mathbb{E}_{\mathbf{x}\sim p_{\text{data}}}[\log D(\mathbf{x})] + \mathbb{E}_{\mathbf{z}\sim p(\mathbf{z})}[\log(1 - D(G(\mathbf{z})))]$$

In practice, the non-saturating generator loss $-\mathbb{E}[\log D(G(\mathbf{z}))]$ is used to avoid gradient saturation early in training.

Intuition — the GAN training dynamic: Think of a counterfeiter (G) and a detective (D). The counterfeiter makes fake currency; the detective tries to spot the fakes. They train against each other: as the detective gets better, the counterfeiter has to improve to keep fooling them, and vice versa. Ideally, the counterfeiter becomes so good that even an expert detective cannot tell real from fake. The generated samples at that point are indistinguishable from real data.

1.5.3 Mode Collapse

Mode collapse occurs when the generator learns to produce only a small variety of outputs — sometimes just one or a few — that reliably fool the discriminator, ignoring the full diversity of the training data distribution.

Cause: The generator finds a "shortcut" — a narrow region of output space where the discriminator consistently fails — and exploits it without exploring further. The discriminator then adapts, but the generator may simply shift to another narrow region rather than covering the full distribution.

Consequences: Generated samples lack diversity; the model fails at tasks requiring varied outputs.

Mitigations: Wasserstein GAN (WGAN) with gradient penalty, minibatch discrimination, unrolled GANs, progressive growing (ProGAN).

1.5.4 Advantages, Disadvantages, and Applications

Aspect	Detail	Notes
Advantages	Generates extremely high-quality, realistic samples	State-of-the-art in image synthesis
	No explicit density model needed	More flexible than VAEs
	Can be conditioned on labels (cGAN)	Useful for controlled generation
Disadvantages	Training instability (oscillating losses)	Requires careful tuning
	Mode collapse	Major unsolved challenge

Aspect	Detail	Notes
	No direct likelihood estimate	Cannot compute $p(\mathbf{x})$
	Evaluation is difficult	FID score used as proxy
Applications	Image synthesis, super-resolution, style transfer, data augmentation, drug discovery, video generation	StyleGAN, CycleGAN, Pix2Pix

1.6 Transfer Learning

Transfer learning is the practice of taking a model pre-trained on a large task and reusing its learned representations for a different (usually smaller or related) task, rather than training from scratch.

1.6.1 Why It Works

Deep networks trained on large datasets (e.g., ImageNet with 1.2M images, 1000 classes) learn general-purpose feature detectors in their early and middle layers — edges, textures, shapes, object parts. These features are useful across many visual tasks. Only the later layers are task-specific.

1.6.2 Two Strategies

Strategy	How it works	When to use
Feature Extraction	Freeze all pre-trained weights. Replace only the final classification head. Train only the new head on target data.	Target dataset is small. Task is similar to source task. Compute is limited.
Fine-Tuning	Unfreeze some or all pre-trained layers. Continue training on target data with a small learning rate.	Target dataset is moderate to large. Task differs enough that some pre-trained features need adjustment.

Practical guidelines:

- If your dataset is very small and similar to the source: use feature extraction only.
- If your dataset is medium-sized and similar: fine-tune the last few layers.
- If your dataset is large but different: fine-tune most or all layers.
- If your dataset is small and very different from the source: transfer learning may not help; consider training from scratch or using data augmentation aggressively.

Intuition: A radiologist who has read 10,000 chest X-rays has learned to recognise textures, shapes, and anomalies in medical images. If you now ask them to read brain MRIs, they are not starting from zero — their low-level visual skills transfer directly. They only need to learn the brain-specific patterns on top. Transfer learning does exactly this: the network already knows how to "see"; you just teach it what to look for in your specific task.

1.6.3 Advantages and Limitations

Advantages: Dramatically reduces training time and data requirements; often achieves better performance than training from scratch on small datasets; leverages compute already spent by large research labs.

Limitations: Source and target domains must be sufficiently similar; negative transfer (performance degradation) can occur if domains are too different; fine-tuning with too large a learning rate can destroy pre-trained features ("catastrophic forgetting").

1.7 CNN and RNN — Specialised Deep Architectures

CNNs and RNNs are specialised extensions of the feedforward DNN, covered in detail in their own units. Their placement in the deep learning landscape:

Architecture	Design motivation	Primary use cases
CNN	Exploits *spatial locality* and *translation invariance* in grid-structured data via shared convolutional filters. Reduces parameters dramatically vs. fully connected layers on images.	Image classification, object detection, segmentation, video
RNN / LSTM	Processes *sequential data* by maintaining a hidden state that is updated at each time step, allowing the network to capture temporal dependencies.	NLP, speech recognition, time series, sequence-to-sequence tasks

Both inherit the deep learning advantage of hierarchical feature learning, but impose structural inductive biases suited to their data modality.

2 Activation Functions

2.1 Why Activation Functions Exist

Without non-linear activation functions, a neural network — regardless of its depth — collapses to a single linear transformation:

$$\mathbf{W}_L(\mathbf{W}_{L-1}(\cdots \mathbf{W}_1\mathbf{x})) = \mathbf{W}_{\text{eff}}\,\mathbf{x}$$

A product of linear transformations is still a linear transformation. No matter how many layers you stack, you can only fit a hyperplane to the data. Activation functions introduce

non-linearity at each neuron, enabling the network to approximate arbitrarily complex functions.

> **Key insight:** Depth without non-linearity is useless. A 100-layer fully linear network is mathematically equivalent to a 1-layer linear model. Activation functions are what make depth meaningful.

2.2 Sigmoid

Formula:

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

Output range: $(0, 1)$

Gradient: $\sigma'(z) = \sigma(z)\,(1 - \sigma(z))$, maximum value 0.25 at $z = 0$.

Characteristics:

- Smooth, differentiable everywhere.
- Outputs are interpretable as probabilities.
- Saturates at both extremes: for large $|z|$, $\sigma'(z) \approx 0$.

Vanishing gradient problem: When $|z|$ is large, the gradient is near zero. During backpropagation, gradients are multiplied across layers. If each layer's gradient ≤ 0.25, multiplying L such terms gives $(0.25)^L$, which vanishes exponentially as L grows. This means early layers receive effectively zero gradient and do not learn.

Use case: Output layer for binary classification. Largely avoided in hidden layers of deep networks.

2.3 Tanh

Formula:

$$\tanh(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}}$$

Output range: $(-1, 1)$

Gradient: $\tanh'(z) = 1 - \tanh^2(z)$, maximum 1.0 at $z = 0$.

Comparison with Sigmoid:

- Tanh is *zero-centred* — outputs are centred around 0, not 0.5. This improves gradient flow in backpropagation because activations are not consistently positive, reducing the "zig-zag" gradient update problem.
- Gradient at the origin is larger (up to 1.0 vs. 0.25 for sigmoid), so gradients die slower.
- Still saturates at ± 1 for large $|z|$; vanishing gradient problem persists in very deep networks.

Use case: Hidden layers of shallow networks, RNN hidden states. Preferred over sigmoid in hidden layers, but still largely superseded by ReLU in feedforward networks.

Intuition — why zero-centering matters: If all activations are positive (as with sigmoid), gradient updates to weights always have the same sign for a given layer, forcing the optimiser to take a "zig-zag" path toward the minimum rather than going directly. Tanh's zero-centred outputs allow both positive and negative gradients, enabling more direct parameter updates.

2.4 ReLU (Rectified Linear Unit)

Formula:

$$\mathrm{ReLU}(z) = \max(0, z)$$

Output range: $[0, +\infty)$

Gradient:

$$\mathrm{ReLU}'(z) = \begin{cases} 1 & z > 0 \\ 0 & z \leq 0 \end{cases}$$

Why ReLU became dominant:

- **No gradient saturation for positive inputs.** Unlike sigmoid/tanh, the gradient is exactly 1 for all $z > 0$, so gradients flow back through active neurons without decay.
- **Computationally trivial.** Just a threshold — one comparison per neuron, no exponentials.
- **Sparse activation.** Approximately half the neurons output zero, creating sparse representations that are computationally efficient and may aid generalisation.
- Empirically, ReLU networks train much faster than sigmoid/tanh networks on deep architectures.

Dying ReLU problem: If a neuron's pre-activation is always negative (due to a large negative bias or a bad weight update), it always outputs zero. The gradient through it is also zero, so the weight never updates — the neuron is permanently "dead" and contributes nothing to learning. This can happen to a significant fraction of neurons, especially with large learning rates.

2.5 Leaky ReLU

Formula:

$$\text{Leaky ReLU}(z) = \begin{cases} z & z > 0 \\ \alpha z & z \leq 0 \end{cases}$$

where α is a small positive constant, typically 0.01.

Fix for dying ReLU: By allowing a small negative slope αz instead of hard-clamping to zero, dying neurons still receive a non-zero gradient and can recover. The neuron is never completely dead.

When to prefer Leaky ReLU: When dying ReLU is observed (many zero activations in training); when training very deep networks; when using aggressive learning rates. The cost is a marginal increase in computation and one extra hyperparameter (α).

Parametric ReLU (PReLU): A variant where α is learned during training rather than fixed.

2.6 ELU (Exponential Linear Unit)

Formula:

$$\mathrm{ELU}(z) = \begin{cases} z & z > 0 \\ \alpha(e^z - 1) & z \leq 0 \end{cases}$$

where $\alpha > 0$ (typically $\alpha = 1$).

Advantages over Leaky ReLU:

- Negative outputs are bounded below by $-\alpha$, so the mean activation is closer to zero than Leaky ReLU — partial zero-centering benefit without needing batch normalisation.
- The smooth exponential transition for $z \leq 0$ avoids the sharp kink at zero, producing smoother gradients.
- Empirically, ELU can train faster and generalise better than ReLU/Leaky ReLU on certain tasks.

Cost: Computing e^z is more expensive than ReLU/Leaky ReLU's simple linear operations.

When to use: When zero-centering of activations matters, when using deeper networks without batch normalisation.

2.7 Softmax

Formula: For a vector $\mathbf{z} = (z_1, z_2, \ldots, z_K)$:

$$\mathrm{Softmax}(z_i) = \frac{e^{z_i}}{\sum_{j=1}^{K} e^{z_j}}, \quad i = 1, \ldots, K$$

Output interpretation: Each output $\mathrm{Softmax}(z_i) \in (0, 1)$ and $\sum_i \mathrm{Softmax}(z_i) = 1$. The outputs form a valid probability distribution over K classes.

Why only at the output layer: Softmax is applied to an entire vector, not element-wise, making it unsuitable for hidden layer neurons (which process independently). At the output layer, it converts raw scores (logits) into class probabilities, enabling cross-entropy loss computation.

Numerical stability: In practice, $\mathrm{Softmax}(z_i) = \frac{e^{z_i - \max(\mathbf{z})}}{\sum_j e^{z_j - \max(\mathbf{z})}}$ is used to prevent overflow from large exponentials.

Softmax vs. Sigmoid for multi-class: Sigmoid applied independently to each output does not guarantee probabilities sum to 1, making it unsuitable for mutually exclusive classes. Softmax enforces this constraint. Use sigmoid multi-output only for *multi-label* classification (where multiple classes can be true simultaneously).

2.8 Full Comparison Table

Function	Formula	Range	Gradient behaviour	Vanish. grad.	Dying neuron	When to use
Sigmoid	$\frac{1}{1+e^{-z}}$	$(0,1)$	Saturates at both ends; max 0.25	High	No	Binary output layer
Tanh	$\frac{e^z-e^{-z}}{e^z+e^{-z}}$	$(-1,1)$	Zero-centred; saturates at ± 1; max 1.0	Moderate	No	RNN hidden states, shallow networks
ReLU	$\max(0,z)$	$[0,\infty)$	Constant 1 for $z>0$; 0 for $z \leq 0$	None (active)	Yes	Default hidden layer choice
Leaky ReLU	$\max(\alpha z, z)$	$(-\infty,\infty)$	α for $z<0$; 1 for $z>0$	None	No	When dying ReLU is a concern
ELU	z or $\alpha(e^z-1)$	$(-\alpha,\infty)$	Smooth; near-zero mean activation	None	No	Deep nets, no batch norm
Softmax	$\frac{e^{z_i}}{\sum e^{z_j}}$	$(0,1)$, sums to 1	Coupled across outputs	—	—	Multi-class output layer only

2.9 Vanishing and Exploding Gradients

Vanishing gradients: During backpropagation, gradients are multiplied through each layer. If the gradient at each layer is < 1, these multiplications drive the gradient toward zero exponentially, making early layers learn extremely slowly or not at all.

Exploding gradients: If the gradient at each layer is > 1, multiplications drive the gradient toward infinity, causing wildly unstable weight updates and divergent training.

2.9.1 Root Cause

The gradient of the loss $\mathcal{L}$ with respect to weights in layer l involves the chain rule product:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{W}^{(l)}} = \frac{\partial \mathcal{L}}{\partial \mathbf{a}^{(L)}} \cdot \prod_{k=l}^{L-1} \frac{\partial \mathbf{a}^{(k+1)}}{\partial \mathbf{a}^{(k)}}$$

Each term in the product includes the activation function's derivative. For sigmoid: max 0.25; for tanh: max 1.0. Multiplying L such terms with values $\ll 1$ causes vanishing; with values $\gg 1$ causes exploding.

2.9.2 Which Activations Make It Worse

- **Sigmoid** — worst offender. Max gradient 0.25; in a 10-layer network, $(0.25)^{10} \approx 10^{-6}$.
- **Tanh** — better but still saturates; gradients vanish in very deep nets.
- **ReLU** — eliminates vanishing for active neurons (gradient exactly 1). Can contribute to exploding if weights are large.

2.9.3 Consequences

- Early layers do not update — the network effectively becomes shallow.
- Training stalls; loss plateaus despite more compute.
- Exploding gradients cause NaN weights; training collapses.

2.9.4 Solutions

- **Use ReLU / Leaky ReLU / ELU** instead of sigmoid/tanh in hidden layers.
- **Proper weight initialisation** (Xavier, He) — discussed in Section 3.
- **Batch normalisation** — normalises activations within each mini-batch, preventing extreme values from propagating.
- **Gradient clipping** — caps gradient magnitude during training; primary defence against exploding gradients.
- **Residual connections (skip connections)** — used in ResNets; allow gradients to flow directly back through shortcut paths, bypassing the multiplicative chain.
- **LSTM/GRU gating** — architectural solution for RNNs; gates control gradient flow over time steps.

3 Hyperparameter Tuning

3.1 Hyperparameters vs. Parameters

Parameters (e.g., weights $\mathbf{W}$ and biases $\mathbf{b}$) are learned automatically from training data via backpropagation and gradient descent.

Hyperparameters are configuration choices set by the practitioner *before* training begins. They control the learning process itself and the architecture, not the model's internal values.

Parameters	**Hyperparameters — Training**	**Hyperparameters — Architecture**
Weights $\mathbf{W}^{(l)}$	Learning rate	Number of layers
Biases $\mathbf{b}^{(l)}$	Batch size	Neurons per layer
Convolutional filters	Number of epochs	Activation function
Embedding vectors	Optimizer type	Dropout rate
	Regularisation strength	Kernel size (CNNs)

3.2 Learning Rate

The **learning rate** η controls the step size of each parameter update:

$$\theta \leftarrow \theta - \eta \cdot \nabla_\theta \mathcal{L}$$

It is widely considered the single most important hyperparameter to tune.

Effect of learning rate:

Learning Rate	Behaviour	Consequence
Too high	Overshoots minimum; loss oscillates or diverges	Training fails; NaN loss
Too low	Very small steps; training is slow	Convergence takes impractically long; may get stuck in poor local minima
Well-tuned	Smooth, steady decrease in loss	Good convergence to a low-loss solution

3.2.1 Learning Rate Schedules

A fixed learning rate is often suboptimal. Common strategies:

- **Step decay:** Reduce η by a factor (e.g., 0.1) every N epochs.
- **Exponential decay:** $\eta_t = \eta_0 \cdot e^{-\lambda t}$.
- **Cosine annealing:** $\eta_t = \eta_{\min} + \frac{1}{2}(\eta_{\max} - \eta_{\min})\left(1 + \cos\frac{t\pi}{T}\right)$.
- **Warmup:** Start with a small η, ramp up over the first k steps, then decay. Used with Adam in Transformers.
- **Cyclical learning rates:** Oscillate η between bounds; can escape saddle points.

3.3 Batch Size

Batch size is the number of training samples used to compute one gradient update.

Batch Size	Advantages	Disadvantages
Small (1–32)	Noisy gradients act as regularisation; can escape sharp minima; lower memory	Slow training; high variance in updates; noisy loss curve
Large (512–4096)	Stable, accurate gradient estimates; faster wall-clock time with GPU parallelism	Often converges to sharp minima that generalise worse; requires more memory; may need η scaling

Batch Size	Advantages	Disadvantages
Medium (64–256)	Practical balance of speed, stability, and generalisation	Still a hyperparameter; task-dependent

The "linear scaling rule": When increasing batch size by a factor k, linearly increase learning rate by k to keep gradient statistics comparable. This is a practical heuristic, not a guarantee.

3.4 Epochs and Early Stopping

Epoch: One complete pass through the entire training dataset.

Choosing the number of epochs is a tuning problem because:

- Too few epochs: underfitting — the model has not had enough time to learn.
- Too many epochs: overfitting — the model memorises training data and validation performance degrades.

Early stopping: Monitor validation loss after each epoch. If validation loss does not improve for p consecutive epochs (patience), stop training and restore the best checkpoint. The number of epochs becomes an implicit hyperparameter determined by validation performance, not set manually.

Advantages: Prevents overfitting; saves compute; implicitly acts as regularisation.

Gotcha: The patience p is itself a hyperparameter. Too small and training stops prematurely on a temporary plateau; too large and overfitting can still occur.

3.5 Number of Layers and Neurons

3.5.1 Depth (Number of Layers)

- More layers enable hierarchical feature learning and greater representational power.
- Too few layers: insufficient capacity, underfitting.
- Too many layers: vanishing gradients, overfitting without sufficient data, slower training.
- Typical practice: start with 2–3 hidden layers for tabular data; go deeper for images/text with residual connections.

3.5.2 Width (Neurons per Layer)

- More neurons per layer increase capacity at each level of abstraction.
- Common heuristics: powers of 2 (64, 128, 256, 512) for memory alignment; wider early layers, narrower later layers.
- Width has diminishing returns beyond a certain point for a given depth.

3.5.3 Depth vs. Width Trade-off

Dimension	Effect	Risk
Increasing depth	Learns more abstract hierarchies; parameter-efficient	Vanishing gradients; harder to optimise
Increasing width	More capacity per level; easier to optimise	More parameters; can overfit; less efficient
Both	Maximum capacity	Extremely prone to overfitting; training instability

Common misconception: Doubling the number of neurons doubles the model's capability. In practice, capacity scales roughly with the number of parameters, but expressive power depends on the interaction between depth and width. A deeper model often achieves the same expressiveness with fewer total parameters.

3.6 Optimizers (Introduction)

Optimisers determine how parameters are updated using computed gradients. The choice of optimiser is a key hyperparameter. Full mathematical treatment is in Unit 4; this section provides the conceptual positioning of each.

Optimizer	Core idea	Best for	When to use
SGD	One update per mini-batch using raw gradient. No adaptive component.	Convex problems; well-tuned large-scale training	When you can tune η manually; CV tasks with careful tuning
Momentum	Accumulates a velocity vector in gradient direction. Dampens oscillations; accelerates along consistent gradient directions.	Non-convex surfaces; escaping local minima	Replacement for vanilla SGD in most cases
RMSProp	Adapts η per-parameter by dividing by an exponentially decaying average of squared gradients. Handles non-stationary objectives.	RNNs; non-stationary loss surfaces	When training is unstable; RNN training

Optimizer	Core idea	Best for	When to use
Adam	Combines momentum (first moment) and RMSProp (second moment) with bias correction.	Most tasks; default starting point	First choice in practice; especially good for NLP, DNNs

Practical advice: Start with Adam. If results are unsatisfactory, try SGD with momentum and a carefully tuned learning rate schedule — SGD sometimes generalises better than Adam on image classification tasks (evidence from ResNet/VGG training).

3.7 Weight Initialization

Weight initialization is the process of setting the initial values of network weights before training begins. It is a hyperparameter in the sense that the initialization scheme is a choice that significantly affects training dynamics.

3.7.1 Why Initialization Matters

Backpropagation multiplies gradients through the weight matrices at each layer. If weights are initialised too large or too small, signals either explode or vanish even at the very first update.

3.7.2 Zero Initialization Problem

Setting all weights to zero is a common beginner mistake. If $\mathbf{W}^{(l)} = 0$ for all l:

- Every neuron in each layer computes the same output.
- Every neuron receives the same gradient during backpropagation.
- All neurons update identically — the network is *symmetric* and can never break symmetry.
- The network is permanently equivalent to a single neuron per layer. Multiple neurons are wasted.

Solution: Always initialise weights with small random values to break symmetry.

Intuition — symmetry breaking: Imagine a class of students who all start with the same answer and receive identical feedback. They will all always think alike and produce the same output. You need students to start with different initial beliefs so that through feedback, they each specialise in something different. Weight initialisation does this for neurons.

3.7.3 Xavier (Glorot) Initialization

Designed for sigmoid and tanh activations. Initialises weights from:

$$W \sim \mathcal{U}\left[-\frac{\sqrt{6}}{\sqrt{n_{\text{in}} + n_{\text{out}}}}, \frac{\sqrt{6}}{\sqrt{n_{\text{in}} + n_{\text{out}}}}\right]$$

or equivalently, a normal distribution with $\sigma^2 = \frac{2}{n_{\text{in}}+n_{\text{out}}}$.

Goal: Keep the variance of activations and gradients approximately equal across layers, preventing signals from growing or shrinking.

3.7.4 He (Kaiming) Initialization

Designed for ReLU and its variants. Initialises weights from:

$$W \sim \mathcal{N}\left(0, \frac{2}{n_{\text{in}}}\right)$$

Why different from Xavier? ReLU zeroes out half its inputs (the negative half), effectively halving the variance. He initialization compensates by using variance $\frac{2}{n_{\text{in}}}$ instead of $\frac{1}{n_{\text{in}}}$.

Method	**Formula**	**Designed for**	**When to use**
Random small	$W \sim \mathcal{N}(0, 0.01)$	Breaking symmetry only	Shallow nets; baseline
Xavier / Glorot	$\sigma^2 = \frac{2}{n_{\text{in}}+n_{\text{out}}}$	Sigmoid, Tanh	When using saturating activations
He / Kaiming	$\sigma^2 = \frac{2}{n_{\text{in}}}$	ReLU, Leaky ReLU	Default for ReLU networks

3.8 Dropout Rate (Introduction)

Dropout is a regularisation technique where, during each training step, each neuron is independently and randomly "dropped" (set to zero) with probability p (the *dropout rate*). At test time, all neurons are active and their outputs are scaled by $(1 - p)$.

Effect: Forces the network to learn redundant representations; prevents co-adaptation of neurons (where one neuron relies on specific other neurons). Acts as implicit ensemble training over exponentially many sub-networks.

Dropout rate as a hyperparameter: Typical values are $p = 0.2$ to $p = 0.5$ for hidden layers; $p = 0.1$ to $p = 0.2$ for input layers. Higher p = stronger regularisation but may underfit.

Full treatment of dropout as a regularisation technique, including its relationship with L1/L2 regularisation and batch normalisation, is covered in Unit 4.

3.9 Hyperparameter Tuning Strategies

With many interacting hyperparameters, an efficient search strategy is necessary.

3.9.1 Grid Search

What it is: Define a discrete set of values for each hyperparameter. Exhaustively try all combinations.

Example: $\eta \in \{0.001, 0.01, 0.1\}$, batch size $\in \{32, 64, 128\} \rightarrow 9$ combinations trained and evaluated.

Advantage	Disadvantage
Exhaustive within specified grid	Exponentially expensive as dimensions increase (curse of dimensionality)
Reproducible; easy to parallelise	Wastes compute if one hyperparameter matters much more than others
Simple to implement	Misses optimal values between grid points

3.9.2 Random Search

What it is: Sample hyperparameter combinations randomly from defined ranges (distributions, not just discrete values) for a fixed budget of trials.

Why it often beats grid search: If one hyperparameter dominates importance, random search will sample it more densely in the important dimension compared to a grid that wastes evaluations on combinations differing only in the irrelevant dimension.

Advantage	Disadvantage
Scales better than grid search with fixed budget	No guarantee of finding optimum
Explores continuous ranges	Results vary between runs (stochastic)
More efficient when some hyperparameters matter more	Can still be wasteful without prior knowledge

Intuition — random vs. grid search: Suppose you're looking for the best coffee-to-water ratio and brewing temperature. If temperature doesn't matter much, a grid wastes trials at the same ratio with slightly different temperatures. A random search will, by chance, try many more ratios — exactly where the answer lies.

3.9.3 Bayesian Optimization

What it is: Maintains a probabilistic surrogate model (typically a Gaussian Process) of the objective function (validation loss as a function of hyperparameters). At each step, uses an *acquisition function* to select the next hyperparameter configuration most likely to improve performance, balancing exploration (untested regions) and exploitation (currently promising regions).

How it works (steps):

1. Evaluate a few initial configurations.
2. Fit a surrogate model to the results (predicts performance for untried configurations).
3. Use the acquisition function (e.g., Expected Improvement) to pick the next configuration.
4. Evaluate, update the surrogate, repeat.

Advantage	**Disadvantage**
Most sample-efficient strategy	Computationally expensive surrogate model fitting
Adapts based on prior results	Complex to implement correctly
Handles continuous, conditional, and high-dimensional spaces	Surrogate model can be misspecified
Often finds better solutions in fewer trials	Parallelisation is non-trivial

3.9.4 Comparison Summary

Strategy	**Efficiency**	**Scalability**	**Adaptiveness**	**Best for**
Grid Search	Low	Poor	None	Small search spaces, few hyperparameters
Random Search	Medium	Good	None	Moderate search spaces; good baseline
Bayesian Optim.	High	Moderate	High	Expensive models; few trial budget